Fred Gabriel – 22511576
Analise e Desenvolvimento de Sistemas
fred.cardoso@sempreceub.com

Especificação Técnica SDD

TaskAnalyzer — Módulo de Análise de Tarefas e Produtividade

1. Visão Geral e Contrato de Negócio (SDD)

1.1 Identificação

Documento produzido individualmente como Fase 1 (Entrega Inicial) do projeto cumulativo do Bootcamp III. Esta especificação será o contrato executável que orientará a geração de código assistida por IA nas Fases 2 e 3.

1.2 Propósito do Módulo TaskAnalyzer

O TaskAnalyzer é um módulo de análise de tarefas e produtividade que recebe um conjunto de tarefas de um usuário ou equipe e calcula métricas objetivas sobre o andamento do trabalho. O sistema deve ser capaz de:

- Calcular o tempo médio de conclusão das tarefas, por tipo/prioridade e de forma geral;
- Identificar a taxa de atraso, isto é, o percentual de tarefas concluídas após o prazo estabelecido;
- Gerar indicadores agregados por prioridade (baixa, média, alta), permitindo priorização baseada em dados;
- Fornecer uma visão consolidada que apoie a tomada de decisão contínua sobre o fluxo de trabalho.

O módulo não é responsável por persistência de dados, autenticação, interface gráfica ou integração com sistemas externos — seu escopo está estritamente limitado ao cálculo determinístico de métricas a partir de uma lista de tarefas fornecida em memória.

1.3 Contrato Executável de Interface

O contrato abaixo define, de forma formal, os tipos de dados, obrigatoriedade dos campos e restrições de negócio que o código gerado por IA deverá seguir estritamente. Nenhum comportamento fora deste contrato deve ser assumido pela IA.

1.3.1 Entradas (Inputs) — Tarefa

Campo	Tipo	Obrigatório	Descrição / Restrições
id_tarefa	int	Sim	Identificador único da tarefa (> 0), não pode se repetir na coleção

Campo	Tipo	Obrigatório	Descrição / Restrições
titulo	str	Sim	Título da tarefa; não pode ser vazio ou apenas espaços em branco
prioridade	str/enum	Sim	Um dos valores: 'baixa', 'media', 'alta' (case-insensitive)
data_criacao	datetime	Sim	Data/hora de criação da tarefa (UTC)
data_inicio	datetime	Não	Data/hora de início da execução, se houver
data_conclusao	datetime	Não	Data/hora de conclusão; obrigatória apenas se status='concluida'
prazo	datetime	Sim	Data/hora limite prevista para conclusão
status	str/enum	Sim	Um dos valores: 'concluida', 'pendente', 'cancelada'

1.3.2 Saídas (Outputs) — Relatório de Métricas

Campo	Tipo	Descrição
tempo_medio_conclusao_min	float	Tempo médio, em minutos, entre data_inicio e data_conclusao das tarefas concluídas
taxa_atraso_percentual	float	Percentual de tarefas concluídas com data_conclusao posterior a prazo, sobre o total de concluídas
quantidade_tarefas	int	Total de tarefas recebidas na análise (todos os status)
indicadores_por_prioridade	dict	Dicionário com métricas (tempo médio, taxa de atraso, quantidade) segmentadas por prioridade (baixa, média, alta)

1.3.3 Regras de Negócio e Restrições

- Somente tarefas com status = 'concluida' entram no cálculo de tempo médio de conclusão e taxa de atraso.
- Tarefas com status = 'cancelada' são contabilizadas em quantidade_tarefas, mas excluídas de todas as métricas de tempo e atraso.
- Uma tarefa com prazo anterior à data_criacao é considerada uma entrada inválida e deve ser rejeitada com uma exceção específica (ver Cenário 2, seção 2.1).
- Divisão por zero (ex.: nenhuma tarefa concluída) deve ser tratada explicitamente, retornando 0.0 e nunca lançando erro de aritmética não tratado.
- Todas as datas devem ser interpretadas em fuso horário UTC; conversões de fuso ficam fora do escopo deste módulo.
- Reaberturas: uma tarefa que retorna de 'concluida' para 'pendente' deve zerar data_conclusao antes de nova análise (regra de consistência, não de cálculo).

2. Especificação de Cenários de Aceite e Test Harness

2.1 Cenários de Aceite (Dado – Quando – Então)

Cenário 1 — Sucesso (cálculo de métricas)

> *Dado um conjunto de tarefas válidas, com prioridades, prazos e status (concluídas e pendentes) consistentes,*
> *Quando o analisador de tarefas for executado sobre essa coleção,*
> *Então o sistema deve retornar as métricas corretas: tempo médio de conclusão, taxa de atraso e quantidade de tarefas corretas por prioridade, sem lançar exceções.*

Cenário 2 — Exceção / Erro (entradas inválidas)

> *Dado uma entrada com datas inválidas (ex.: prazo anterior à data de criação) ou uma condição de divisão por zero (nenhuma tarefa concluída ao calcular médias),*
> *Quando o analisador de tarefas for executado,*
> *Então o sistema deve disparar uma exceção específica e tratada (ex.: TarefaInvalidaError), com mensagem clara e objetiva ao usuário, sem interromper o processo de forma não controlada.*

Cenário 3 — Borda (coleção vazia)

> *Dado uma lista de tarefas vazia,*
> *Quando o analisador de tarefas for executado,*
> *Então o sistema deve retornar um relatório de métricas com quantidade_tarefas = 0, tempo_medio_conclusao_min = 0.0, taxa_atraso_percentual = 0.0 e indicadores_por_prioridade vazio, sem lançar exceção.*

2.2 Planejamento do Test Harness

Os cenários de aceite acima serão convertidos em testes automatizados com pytest na Fase 2, formando o test harness que valida objetivamente o código gerado pela IA. O fluxo planejado é:

1. Cenários de Aceite (este documento) descrevem o comportamento esperado do sistema em linguagem Dado–Quando–Então e servem como referência única para os testes.
2. Casos de Teste decompõem cada cenário em casos específicos (ex.: tarefa concluída no prazo, tarefa concluída em atraso, tarefa pendente, tarefa cancelada, lista vazia, prazo inválido).
3. Testes Automatizados (pytest) implementam cada caso de teste usando fixtures para preparar dados de entrada e asserts para validar saídas e exceções esperadas.
4. Execução e Validação: os testes serão executados de forma contínua a cada geração/alteração de código pela IA, antes de qualquer aceite.
5. Relatórios e Evidências: cada execução gera um relatório de cobertura e resultados, garantindo rastreabilidade entre requisito, cenário e teste.

Objetivo: garantir que o código gerado pela IA esteja correto, em total conformidade com o contrato de negócio (seção 1.3) e com os cenários de aceite definidos (seção 2.1).

3. Governança de Contexto e Regras para Agentes de IA (CONTEXT_RULES)

As regras abaixo comporão o arquivo CONTEXT_RULES.md, que será a fonte da verdade fornecida aos agentes de IA na Fase 2, garantindo alinhamento, consistência e qualidade do código gerado.

3.1 Diretrizes Arquiteturais (Obrigatórias)

- Utilizar Python 3.11 ou versão superior como mínimo obrigatório.
- Utilizar type hints (anotações de tipo) em todas as funções e métodos.
- Seguir o princípio Single Responsibility (SRP) e boas práticas de código limpo (PEP 8).
- Documentação formal no padrão Google style docstrings para módulos, classes, funções e parâmetros.
- Funções devem ser pequenas, coesas e com um único propósito.
- Tratamento de exceções específico, com classes de exceção próprias (ex.: TarefaInvalidaError) e mensagens claras.
- Logs estruturados utilizando o módulo padrão logging, sem uso de print para depuração.
- Código deve ser legível, manutenível e com nomes de variáveis e funções descritivos, em português ou inglês de forma consistente em todo o projeto.

3.2 Proibições Explícitas (Regras Restritivas)

- Não utilizar bibliotecas externas não autorizadas (somente biblioteca padrão do Python e pytest para testes).
- Não alterar, remover ou adaptar os cenários de teste fornecidos na seção 2.1 deste documento.
- Não modificar a estrutura de pastas definida na seção 4.1 desta especificação.
- Não persistir dados em arquivos ou bancos de dados — o módulo opera inteiramente em memória.
- Não alterar a assinatura (nome e parâmetros) das funções públicas sem autorização explícita registrada.
- Não gerar código sem os testes automatizados correspondentes para novas funcionalidades.
- Não inserir código duplicado ou desnecessário (evitar over-engineering).
- Não assumir comportamentos não especificados neste contrato de negócio.

3.3 Regras de Interação com a IA

- Fornecer sempre o contexto completo (este documento + CONTEXT_RULES.md) antes de solicitar geração de código.
- Validar se a IA compreendeu o contrato antes de gerar a solução, solicitando um resumo do entendimento quando necessário.
- Solicitar explicações sobre qualquer decisão de design tomada pela IA que não esteja explicitamente coberta por esta especificação.
- Revisar criticamente todo código gerado antes de aceitá-lo (nenhum código é versionado sem homologação humana, conforme seção 4.2).
- Rejeitar e solicitar nova geração para respostas que violem qualquer proibição da seção 3.2 ou o contrato de negócio da seção 1.3.

4. Arquitetura do Repositório e Preparação para Git/GitHub

4.1 Desenho da Árvore do Repositório

Estrutura de arquivos e pastas que será versionada no GitHub a partir da Fase 2, já definida nesta especificação para orientar o assistente de IA:

```
sdd-task-analyzer/
├── README.md                   # Visão geral do projeto no GitHub
├── CONTEXT_RULES.md            # Regras persistentes para a IA (seção 3)
├── specs/
│   └── task_analyzer_spec.md   # Especificação SDD (baseada neste documento)
├── tests/
│   └── test_harness.py         # Testes automatizados de validação (pytest)
├── src/
│   └── task_analyzer.py        # Código gerado via IA e homologado
├── .gitignore                  # Arquivos e pastas ignoradas
└── requirements.txt            # Dependências (pytest)
```

Estratégia de versionamento planejada para a Fase 2: fluxo Git simplificado (main / develop / feature/*), commits pequenos e descritivos, Pull Requests para toda alteração revisada, revisões obrigatórias antes de merge e tags para releases importantes.

4.2 Plano de Homologação Humana

Na Fase 2, todo código gerado pela IA será submetido a uma revisão crítica humana antes de ser aceito e versionado. A homologação seguirá os seguintes passos:

1. Execução completa dos testes automatizados (test harness) para validação funcional.
2. Revisão de conformidade com o contrato de negócio (seção 1.3) e com o CONTEXT_RULES.md (seção 3).
3. Análise de qualidade de código: legibilidade, tratamento de erros e aderência aos padrões definidos (seção 3.1).
4. Testes manuais complementares em casos de borda não cobertos automaticamente.
5. Somente após aprovação explícita, o código é commitado no repositório.

Esse processo garante que a IA seja uma ferramenta produtiva, mas que a decisão final e a responsabilidade técnica permaneçam humanas — em conformidade com o princípio de homologador crítico descrito no papel do desenvolvedor moderno.

5. Roteiro do Vídeo de Apresentação (5 minutos)

Vídeo a ser gravado com participação obrigatória do estudante em câmera e áudio, na horizontal, publicado no YouTube como Público ou Não Listado (não pode ser Shorts).

Bloco 1 — Apresentação (≈ 30s)

- Nome completo, curso e motivação para o Bootcamp III.

Bloco 2 — Mostrar a especificação (≈ 2min30)

- Apresentar o documento no Google Docs: visão geral, propósito do TaskAnalyzer e contrato executável de interface.
- Explicar os cenários de aceite (Dado–Quando–Então) e como se transformarão em testes automatizados na Fase 2.

Bloco 3 — Explicar regras e estrutura (≈ 2min)

- Apresentar as diretrizes arquiteturais e proibições explícitas do CONTEXT_RULES.
- Mostrar a árvore de repositório planejada para o GitHub e o plano de homologação humana.

Link do vídeo de apresentação

Link ativo (YouTube — Público ou Não Listado): [inserir link aqui antes do envio]

> **Checklist final antes do envio**
>
> ☐ Documento no Google Docs com título [Especificacao SDD TaskAnalyzer - Seu Nome Completo]
> ☐ Identificação preenchida (nome, curso, polo/turma, e-mail institucional)
> ☐ Vídeo gravado (5 min, horizontal, com rosto e áudio), publicado como Público ou Não Listado
> ☐ Link do vídeo inserido no documento
> ☐ Exportação em PDF enviada no AVA junto com o link do Google Docs